

# Introductions to Big Data Systems: Homework 10

Francisco Diego Ariel Romero Contreras

法兰西思科

francisco.romero.c@ug.uchile.cl

2025403471

## Isomorphism

### Format of Isomorphism

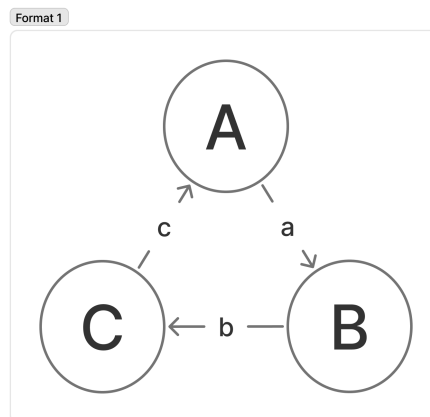


Figure 1: Close Cycle Isomorphism.

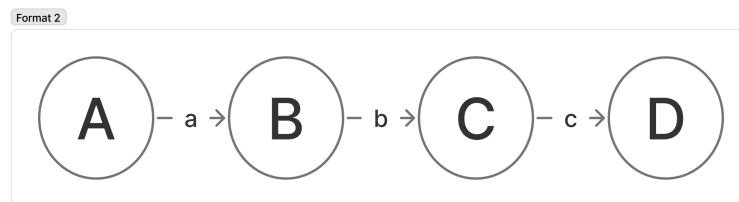


Figure 2: Linear Isomorphism.

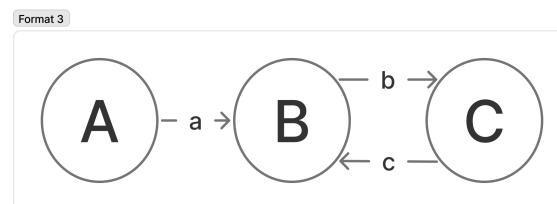


Figure 3: Open Cycle Isomorphism.

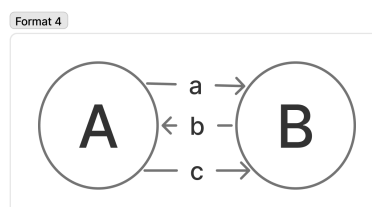


Figure 4: Dual Vertex Close Cycle Isomorphism.

## Concrete isomorphism

There is a modification on the json definition, where are added the labels “format” and “vertex\_id\_ref”. “format” define the kind of abstract isomorphism of the pattern. “vertex\_id\_ref” says if the the real identification of the vertex, so when there is a cycle, different vertex can have the same reference.

For the inform data was used: ./FSM data 10000 1000000 5000000.

The output of the call was:

Total patterns enumerated: 616743645  
Unique patterns found: 2643452  
Found 234 patterns with support >= 10000

The top ten more frequent isomorphism are format 2.

### 1. Frequency: 53759

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode2
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

### 2. Frequency: 33501

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is John
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3

- business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode2
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

### 3. Frequency: 32809

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is John
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode2
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

### 4. Frequency: 32419

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is John
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3

- business code is buscode2
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

5. Frequency: 32311

- Format: 2
- Vertex 0
  - Name is John
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode2
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

6. Frequency: 27684

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3

- business code is buscode3

7. Frequency: 27254

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-2
  - business code is buscode3
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

8. Frequency: 27083

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-2
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

9. Frequency: 27068

- Format: 2

- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode1
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

10. Frequency: 27025

- Format: 2
- Vertex 0
  - Name is Mike
- Vertex 1
  - Name is Mike
- Vertex 2
  - Name is Mike
- Vertex 3
  - Name is Mike
- Edge a
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode2
- Edge b
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3
- Edge c
  - amount of money is 0.0
  - strategy is strategy\_name-3
  - business code is buscode3

For a further analysis some other patten are:

Type 3:

```
{
  "frequency": 13014,
  "format": 3,
  "nodes": [
    {
      "node_id": 0,
      "node_id_ref": 0,
      "name": "Mike"
    },
    {
      "node_id": 1,
      "node_id_ref": 1,
      "name": "Mike"
    },
    {
      "node_id": 2,
      "node_id_ref": 0,
      "name": "Mike"
    },
    {
      "node_id": 3,
      "node_id_ref": 2,
      "name": "Mike"
    }
  ],
  "edges": [
    {
      "source_node_id": 0,
      "target_node_id": 1,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode1"
    },
    {
      "source_node_id": 1,
      "target_node_id": 2,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode2"
    },
    {
      "source_node_id": 2,
      "target_node_id": 3,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode3"
    }
  ]
},
```

For another input can be found some other types of format.

Type 4:

```
{
  "frequency": 7091,
  "format": 4,
  "nodes": [
    {
      "node_id": 0,
      "node_id_ref": 0,
      "name": "Mike"
    },
    {
      "node_id": 1,
      "node_id_ref": 0,
      "name": "Mike"
    },
    {
      "node_id": 2,
      "node_id_ref": 1,
      "name": "Mike"
    },
    {
      "node_id": 3,
      "node_id_ref": 1,
      "name": "Mike"
    }
  ],
  "edges": [
    {
      "source_node_id": 0,
      "target_node_id": 1,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode2"
    },
    {
      "source_node_id": 1,
      "target_node_id": 2,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode3"
    },
    {
      "source_node_id": 2,
      "target_node_id": 3,
      "amt": 0.0,
      "strategy_name": "strategy_name-3",
      "buscode": "buscode3"
    }
  ]
},
```



## Data structure

```
struct Vertex{
    VertexID id;
    std::string name;
    int64_t timestamp;
    int black;
};

struct Edge{
    VertexID source_id;
    VertexID target_id;
    int64_t timestamp;
    double amt;
    std::string strategy_name;
    std::string buscode;
    int64_t trade_no;
};

// Lightweight edge for efficient storage
struct LightEdge {
    VertexID source;
    VertexID target;
    double amt;
    std::string strategy_name;
    std::string buscode;
};

struct Graph{
    std::unordered_map<VertexID, Vertex> vertexMap;
    std::vector<LightEdge> edges;
    std::unordered_map<VertexID, std::vector<int>> outAdj; // adjacency list: vertex
-> edge indices
    std::unordered_map<VertexID, std::vector<int>> inAdj; // reverse adjacency list
};
```

## Algorithm

To compile should be used this bash line:

```
g++ -O3 -fopenmp graph.h graph.cpp FSM.cpp -o FSM
```

To run the algorithm:

```
./FSM <graph path> [<support>] [<Maximum of vertex>] [<Maximum number of patten
saved>]
```

## Pseudo-code

### Enumerate 3-Edge Patterns

```
1  Given Graph g
2  compute degreeList from all vertices
3  sort degreeList by total-degree descending
4  set topK to min(HIGHEST_DEGREE, |degreeList|)
5  parallel for v_idx in [0 .. topK)
6      v ← degreeList[v_idx].vertex
7      inEdges ← g.inAdj[v]
8      outEdges ← g.outAdj[v]
9      for each e1 in inEdges
10         u ← e1.source
11         for each e2 in outEdges
12             w ← e2.target
13             if (u, v) ≥ (v, w) then
14                 | continue to next edge
15             end
16             wOut ← g.outAdj[w]
17             for each e3 in wOut
18                 x ← e3.target
19                 if (v, w) ≥ (w, x) then
20                     | continue to next edge
21                 end
22                 canonical ← getCanonicalForm(g, u, v, w, x, e1, e2, e3)
23                 localPatterns[canonical] ← localPatterns[canonical] + 1
24             end
25         end
26     end
27 Merge from each localPatterns to globalPatterns
```

## Canonical Form

```
1 Collect input vertex IDs: u, v, w, x into list vids
2 Prepare vector length 4 labels
3 cycle  $\leftarrow$  2 //kind of format
4 if u equals x then
5   | cycle  $\leftarrow$  1
6 else if u equals w and v equals x then
7   | cycle  $\leftarrow$  3
8 else if u equals w or v equals x then
9   | cycle  $\leftarrow$  4
10 end
11 if cycle distinct 2 then
12   | canon  $\leftarrow$  0
13   | for each i in 0..3 do
14     | if count of i on vid2canon is equals 0 then
15       | vid2canon[i]  $\leftarrow$  canon;
16       | canon  $\leftarrow$  canon + 1
17     | Add { label: g.vertexMap[vid2canon[i]].name, idx: i } to vint
18   | end
19 else
20   | for each i in 0..3 do
21     | Add { label: g.vertexMap[vids[i]].name, idx: i } to vint
22   | end
23 end
24 Add vint to canonical
25 Prepare edges list edges
26 Define helper addEdge(e, srcIdx, tgtIdx):
27   Add { s: canonOf[srcIdx], t: canonOf[tgtIdx], strategy: e.strategy_name, buscode: e.buscode,
    amt: round(e.amt) } to edges
28 Add edges: e1(0,1), e2(1,2), e3(2,3)
29 Sort edges canonically by strategy, buscode, amt, s, t
30 Build canonical string form
31 Append vint labels to form
32 Append cycle to form
33 Append edges labels to form
34 Return form
```

## **Main**

- 1 load Graph G from dataPath
- 2 Create empty map patterns
- 3 Enumerate 3-Edge patterns of G
- 4 Write Patterns in a JSON

## **Optimizations**

### **Naive edge de-duplicate**

As the search can arrive to the pattern (e1, e2, e3) and (e2, e1, e3) in different time, is necessary a de-duplicate method, so the algorithm only take a order where:  $e1 \geq e2 \geq e3$ .

### **Top degrees**

One optimization is the top degree, where only are compared the top degree vertex, this allow run a lower subset of the dataset, this implementation is optional, if the HIGHEST\_DEGREE is bigger that the amount of vertex then use all the dataset.

### **Max pattern**

This optimization restrain the number of pattern saved on memory for each thread, this method avoid saving on storage all the patterns, this approximation is made to avoid using external storage but doesn't count all the edges.